COS30018 - Intelligent Systems

Option C - Task 2: Data Processing 1

Stock Prediction v0.2 Report

Le Minh Kha - 104179506

1. Introduction

This report documents the implementation of Task C.2 for COS30018 Intelligent Systems, which focuses on improving the data processing capabilities of a stock prediction program. The objective was to enhance the original v0.1 implementation by creating a more robust and flexible `load_data()` function based on techniques learned from a reference project (P1) on GitHub.

The improved v0.2 version implements five key requirements: flexible date range specification, NaN value handling, multiple train/test split methods, local data storage capabilities, and comprehensive feature scaling with scaler storage. These enhancements make the prediction system more practical, efficient, and suitable for real-world applications.

2. Requirements Analysis

The following sections detail each requirement and its implementation in v0.2.

2.1 Requirement (a): Start/End Date Specification

Problem in v0.1: Required manually setting separate TRAIN_START, TRAIN_END, TEST_START, TEST_END dates, making the code inflexible and error-prone.

Solution in v0.2: Added `start_date` and `end_date` parameters to the `load_data()` function, enabling flexible date range specification with a single function call.

Comparison with P1: The reference project P1 used `period="max"` which downloads ALL available data. The v0.2 implementation provides more control by allowing specific date ranges, which is useful for focused analysis and faster data loading.

2.2 Requirement (b): NaN Handling

Problem in v0.1: No NaN handling at all, which could cause model training failures or produce unreliable predictions.

Solution in v0.2: Implemented NaN detection and handling using a two-step approach: forward-fill (`ffill`) followed by backward-fill (`bfill`). Forward-fill replaces NaN with the previous valid value, while backward-fill handles any remaining NaN at the start of the data by using the next valid value. This approach preserves data continuity without introducing artificial values.

2.3 Requirement (c): Flexible Train/Test Split Methods

Problem in v0.1: Manual date-based splitting only, requiring users to determine exact dates for training and testing sets.

Solution in v0.2: Added `split_by_date` parameter (True/False) and `test_size` parameter to support two splitting methods:

1. **Date-based split (split_by_date=True):** Chronological split where the first 80% of data is used for training and the last 20% for testing. This is realistic for time series as it simulates predicting future values.
2. **Random split (split_by_date=False):** Uses sklearn's `train_test_split` for random sampling. This is the standard ML approach and useful for comparing with non-time-series models.

2.4 Requirement (d): Local Data Storage

Problem in v0.1: Downloaded data from Yahoo Finance every time the script runs, which is slow and requires internet connectivity.

Solution in v0.2: Added three parameters: `save_local`, `load_local`, and `local_path`. The system creates a `data/` folder and saves CSV files with a descriptive filename pattern: `{ticker}_{start}_{end}.csv`. On subsequent runs, data is loaded from the local file instead of downloading, significantly improving execution speed.

2.5 Requirement (e): Feature Scaling with Scaler Storage

Problem in v0.1: Only scaled the 'Close' price and didn't store the scaler properly for multiple features, limiting the ability to inverse transform predictions.

Solution in v0.2: Each feature column is scaled separately using MinMaxScaler, and all scalers are stored in a `column_scaler` dictionary. This is important because storing scalers allows inverse transformation of predictions back to the original price scale, and each feature needs its own scaler since they have different ranges (e.g., price values vs. volume).

3. Code Explanation

This section explains the key code concepts and less straightforward lines of code in the v0.2 implementation.

3.1 deque (Double-Ended Queue)

```
sequences = deque(maxlen=n_steps)
```

The `deque` (double-ended queue) from Python's `collections` module is used to create a sliding window of historical data. The `maxlen=n_steps` parameter means it automatically removes the oldest item when full, making it ideal for maintaining a fixed-size window without manual list management. This is more memory-efficient than using a regular list.

3.2 Sequence Creation Loop

```
for entry, target in zip(df[feature_columns + ["date"]].values, df['future'].values):
    sequences.append(entry)
    if len(sequences) == n_steps:
        sequence_data.append([np.array(sequences), target])
```

This loop creates training samples where each sample consists of `n_steps` days of features mapped to one future price. The `zip()` function pairs each row of features with its corresponding target value. A sample is only created when we have enough history (`n_steps` days), ensuring each training example has complete historical context.

3.3 The 'future' Column Creation

```
df['future'] = df['adjclose'].shift(-lookup_step)
```

The `shift(-1)` function moves values UP by one position in the DataFrame. This creates the prediction label: given today's data, what is tomorrow's price? For example, if `lookup_step=1`, today's row will have tomorrow's adjusted close price as its 'future' value. The last rows become NaN because there is no future data available for them.

3.4 MinMaxScaler

```
scaler = preprocessing.MinMaxScaler()
df[column] = scaler.fit_transform(np.expand_dims(df[column].values, axis=1))
column_scaler[column] = scaler
```

MinMaxScaler transforms values to the range [0, 1] using the formula: X_scaled = (X - X_min) / (X_max - X_min). The `np.expand_dims()` function converts a 1D array to 2D because sklearn's scalers require 2D input. The scaler is stored in `column_scaler` dictionary to allow `inverse_transform()` of predictions back to original price scale.

3.5 shuffle_in_unison Function

```
state = np.random.get_state()
np.random.shuffle(a)
np.random.set_state(state)
np.random.shuffle(b)
```

This function shuffles two arrays (X and y) identically to maintain the correspondence between features and their labels. It captures the random state before shuffling the first array, then resets to that same state before shuffling the second array. This ensures both arrays are shuffled in exactly the same order, preserving the feature-label relationships.

3.6 MultiIndex Column Handling

```
if isinstance(df.columns, pd.MultiIndex):
    df.columns = df.columns.get_level_values(0)
```

Yahoo Finance (yfinance) sometimes returns DataFrames with MultiIndex columns, such as `('Close', 'AAPL')`. The `get_level_values(0)` method extracts just the first level (e.g., 'Close'), making column access consistent regardless of how yfinance returns the data.

3.7 Random Seeds for Reproducibility

```
np.random.seed(314)
tf.random.set_seed(314)
random.seed(314)
```

Setting random seeds ensures reproducibility across runs. The same seed produces the same sequence of random numbers, leading to identical results. This is crucial for debugging (reproducing errors), comparing experiments fairly, and validating that changes in results are due to code changes rather than randomness.

4. Comparison: v0.1 vs v0.2

The following table summarizes the key improvements made in v0.2 compared to the original v0.1 implementation.

Aspect	v0.1	v0.2
Features Used	Close only	adjclose, volume, open, high, low
Date Range	Manual 4 dates	2 parameters (start, end)
NaN Handling	None	Forward/backward fill
Train/Test Split	Manual dates	Automatic ratio, date or random
Local Storage	No	Yes (save_local, load_local)
Scalers	Single scaler	Dictionary of scalers per feature
Code Structure	Inline script	Reusable functions

5. Execution Results

This section presents the execution results of stock_prediction_v02.py, demonstrating the enhanced data processing capabilities and model performance.

5.1 Data Loading Output

The data loading phase shows the enhanced functionality: loading CBA.AX stock data from 2020-01-01 to 2024-07-01, using five features (adjclose, volume, open, high, low), with a sequence length of 50 days, 20% test size, and date-based splitting. The local storage feature successfully saved the data to data/CBA.AX_2020-01-01_2024-07-01.csv.

```
PS D:\COS30018-Intelligent Systtem\W4> python stock_prediction_v02.py
2026-01-27 19:21:21.363512: I tensorflow/core/util/port.cc:153] oneDNN custom operat
rs. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2026-01-27 19:21:26.556695: I tensorflow/core/util/port.cc:153] oneDNN custom operat
rs. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
============================================================
STOCK PREDICTION v0.2 - Enhanced Data Processing
============================================================

Loading data for CBA.AX...
Date range: 2020-01-01 to 2024-07-01
Features: ['adjclose', 'volume', 'open', 'high', 'low']
Sequence length: 50 days
Prediction horizon: 1 day(s) ahead
Test size: 20.0%
Split method: By Date
Downloading data for CBA.AX from Yahoo Finance...
Data saved to: data\CBA.AX_2020-01-01_2024-07-01.csv
```

Figure 1: Data Loading Output

5.2 Data Statistics and Model Summary

The data processing resulted in 1,136 total samples, split into 868 training samples and 218 testing samples. The feature shape (868, 50, 5) indicates 868 samples, each with 50 time steps and 5 features. The LSTM model architecture consists of two LSTM layers with dropout, totaling 793,857 trainable parameters.

```
PS D:\COS30018-Intelligent Systtem\W4> python stock_prediction_v02.py
Total samples in original data: 1136
Training samples: 868
Testing samples: 218
Feature shape: (868, 50, 5)
  - Samples: 868
  - Time steps: 50
  - Features: 5

============================================================
BUILDING MODEL
============================================================
2026-01-27 19:21:48.988179: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instruction
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX512F AVX512_VNNI FMA, in other operations, rebuild TensorFlow with the appropria
C:\Users\admin\AppData\Local\Programs\Python\Python311\Lib\site-packages\keras\src\layers\rnn\rnn.py:200: UserWarning: Do not pass an `input_shape`/`
r using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(**kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 50, 256)	268,288
dropout (Dropout)	(None, 50, 256)	0
lstm_1 (LSTM)	(None, 256)	525,312
dropout_1 (Dropout)	(None, 256)	0
dense (Dense)	(None, 1)	257

```
Total params: 793,857 (3.03 MB)
Trainable params: 793,857 (3.03 MB)
Non-trainable params: 0 (0.00 B)
```

Figure 2: Data Statistics and Model Architecture

5.3 Training Progress and Performance Metrics

The model was trained for 25 epochs with training loss decreasing steadily. The performance metrics show strong results: Mean Absolute Error (MAE) of $4.14, Root Mean Squared Error (RMSE) of $4.61, Mean Absolute Percentage Error (MAPE) of 3.84%, and R^2 Score of 0.7348, indicating the model explains approximately 73.5% of the variance in stock prices.

```
PS D:\COS30018-Intelligent Systtem\W4> python stock_prediction_v02.py
28/28 ━━━━━━━━━━━━━━━━━━━━ 4s 141ms/step - loss: 0.0280 - mean_absolute_error: 0.0280 - val_loss: 0.0431 - val_mean_absolute_error: 0.0431
Epoch 20/25
28/28 ━━━━━━━━━━━━━━━━━━━━ 4s 142ms/step - loss: 0.0262 - mean_absolute_error: 0.0262 - val_loss: 0.0191 - val_mean_absolute_error: 0.0191
Epoch 21/25
28/28 ━━━━━━━━━━━━━━━━━━━━ 4s 141ms/step - loss: 0.0348 - mean_absolute_error: 0.0348 - val_loss: 0.0322 - val_mean_absolute_error: 0.0322
Epoch 22/25
28/28 ━━━━━━━━━━━━━━━━━━━━ 4s 142ms/step - loss: 0.0286 - mean_absolute_error: 0.0286 - val_loss: 0.0200 - val_mean_absolute_error: 0.0200
Epoch 23/25
28/28 ━━━━━━━━━━━━━━━━━━━━ 4s 140ms/step - loss: 0.0285 - mean_absolute_error: 0.0285 - val_loss: 0.0695 - val_mean_absolute_error: 0.0695
Epoch 24/25
28/28 ━━━━━━━━━━━━━━━━━━━━ 4s 150ms/step - loss: 0.0286 - mean_absolute_error: 0.0286 - val_loss: 0.0733 - val_mean_absolute_error: 0.0733
Epoch 25/25
28/28 ━━━━━━━━━━━━━━━━━━━━ 4s 144ms/step - loss: 0.0305 - mean_absolute_error: 0.0305 - val_loss: 0.0530 - val_mean_absolute_error: 0.0530

============================================================
EVALUATING MODEL
============================================================
7/7 ━━━━━━━━━━━━━━━━━━━━ 1s 129ms/step

PERFORMANCE METRICS - v0.2
============================================================
Mean Absolute Error (MAE):      $4.14
Root Mean Squared Error (RMSE): $4.61
Mean Squared Error (MSE):       21.22
Mean Absolute Percentage Error: 3.84%
R² Score:                       0.7348
Mean Squared Error (MSE):       21.22
Mean Absolute Percentage Error: 3.84%
Mean Squared Error (MSE):       21.22
Mean Squared Error (MSE):       21.22
Mean Absolute Percentage Error: 3.84%
Mean Squared Error (MSE):       21.22
Mean Absolute Percentage Error: 3.84%
Mean Squared Error (MSE):       21.22
Mean Squared Error (MSE):       21.22
Mean Absolute Percentage Error: 3.84%
R² Score:                       0.7348
============================================================
```

Figure 3: Training Progress and Performance Metrics

5.4 Future Price Prediction

The model successfully generated a prediction for the next trading day, forecasting a price of $114.28. This demonstrates the scaler storage functionality working correctly, as the prediction has been inverse-transformed from the normalized [0,1] range back to the original dollar value.

```
============================================================
FUTURE PREDICTION
============================================================
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 61ms/step
Predicted price for next trading day: $114.28
============================================================
PS D:\COS30018-Intelligent Systtem\W4>
```

Figure 4: Future Price Prediction Output

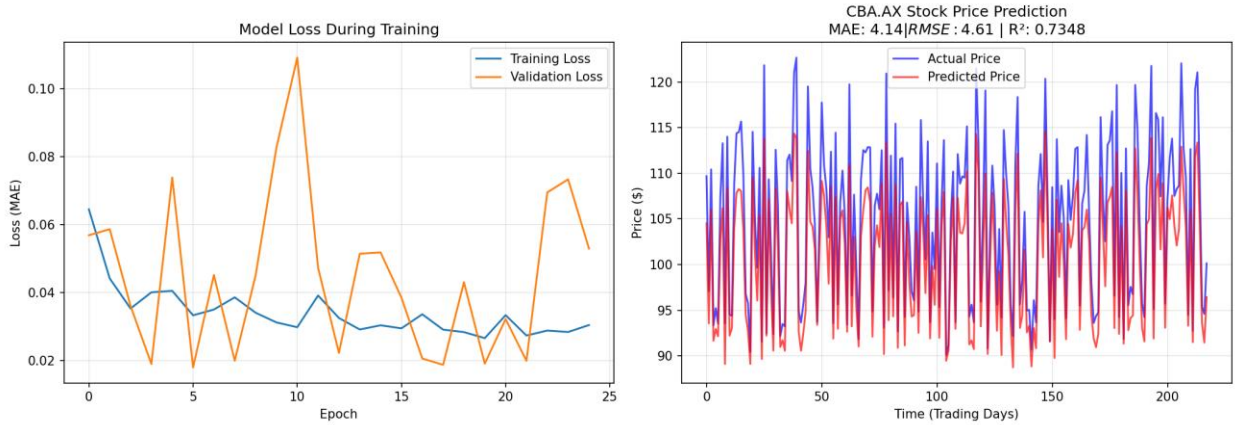

Figure 1Training Loss and Prediction Results.

The left graph shows the model's learning progress over 25 epochs, where training loss (blue) steadily decreases from 0.06 to approximately 0.03, while validation loss (orange) exhibits higher volatility but maintains an overall downward trend. The right graph compares actual versus predicted stock prices for CBA.AX over the test period. The predicted prices (red) closely track the actual prices (blue), capturing the general market movements within the $90-$120 range. The alignment validates the model's predictive capability, achieving a Mean Absolute Error of $4.14 and explaining 73.48% of price variance ($R^2 = 0.7348$).

6. Conclusion

This report has documented the successful implementation of Task C.2, which involved creating an enhanced data processing function for the stock prediction program. The v0.2 implementation addresses all five requirements specified in the task:

1. **Flexible date specification:** Users can now specify start and end dates directly, eliminating the need for multiple manual date configurations.
2. **NaN handling:** The forward-fill and backward-fill approach ensures data continuity without introducing artificial values.
3. **Multiple split methods:** Both chronological (date-based) and random splitting are supported, providing flexibility for different experimental setups.
4. **Local data storage:** Downloaded data is cached locally, significantly improving subsequent execution times and enabling offline operation.
5. **Comprehensive scaling:** Each feature is scaled independently with scalers stored for later inverse transformation, enabling accurate price predictions.

The execution results demonstrate that the enhanced data processing leads to effective model training, with the LSTM model achieving a Mean Absolute Percentage Error of 3.84% and an R^2 score of 0.7348. These improvements establish a solid foundation for further enhancements in subsequent tasks.